

Consultas Avançadas: Junções e Subconsultas

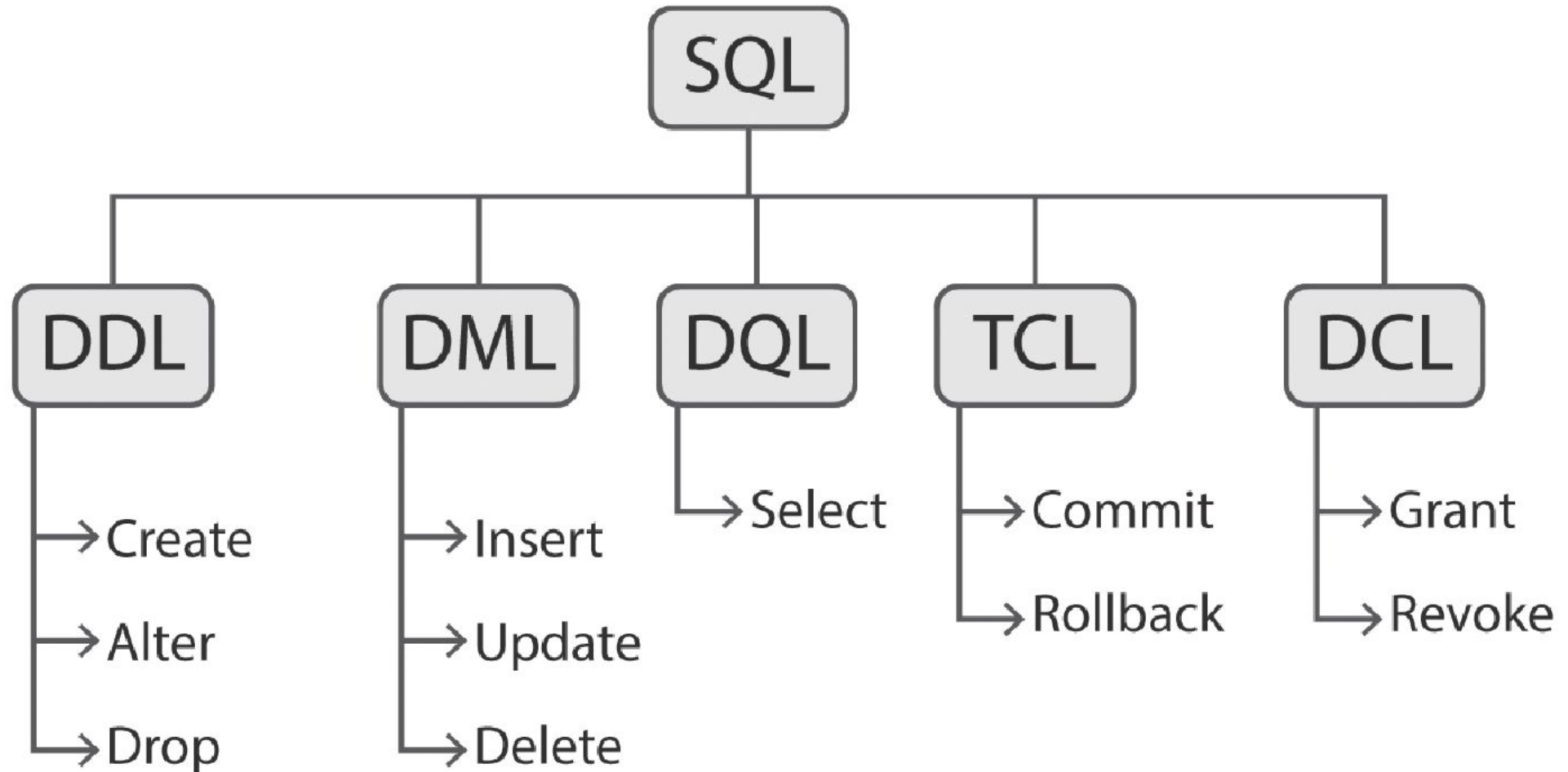
Prof. Cloves Rocha

Ciência da Computação - CC

Roteiro

- DQL - *Data Query Language*.
- Junções
- Exercícios.

DQL



Junções

- Uma cláusula **JOIN** em SQL, correspondente a uma operação de junção em álgebra relacional, combina colunas de uma ou mais tabelas em um banco de dados relacional.
- Ela cria um conjunto que pode ser salvo como uma tabela ou usado da forma como está.
- Um **JOIN** é um meio de combinar colunas de uma ou mais tabelas, usando valores comuns a cada uma delas.
- O SQL padrão **ANSI** especifica cinco tipos de **JOIN**: **INNER JOIN**, **LEFT JOIN**, **RIGHT JOIN**, **FULL JOIN** e **CROSS JOIN**.

Junções

Principais tipos:

- [INNER] JOIN
- LEFT [OUTER] JOIN
- RIGHT [OUTER] JOIN
- CROSS JOIN

Variações mais comuns:

- FULL [OUTER] JOIN

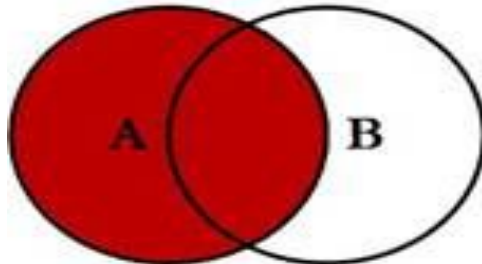
Junções

Outras variações:

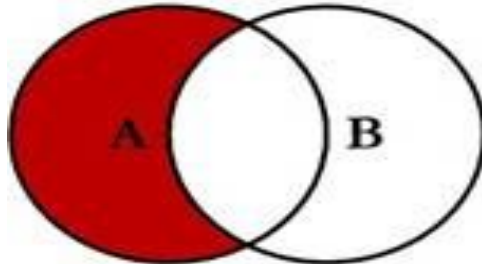
- LEFT [OUTER EXCLUDING] JOIN
- RIGHT [OUTER EXCLUDING] JOIN
- FULL [OUTER EXCLUDING] JOIN

Junções

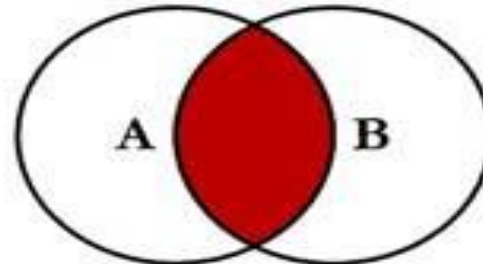
SQL JOINS



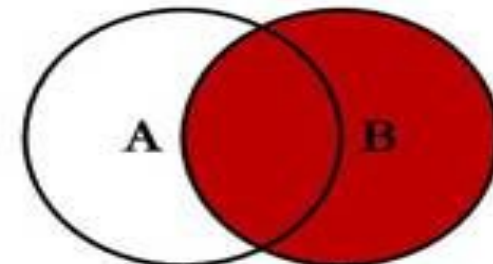
```
SELECT <select_list>  
FROM TableA A  
LEFT JOIN TableB B  
ON A.Key = B.Key
```



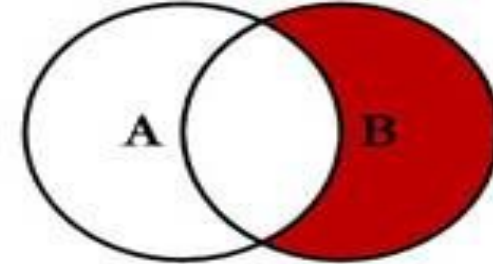
```
SELECT <select_list>  
FROM TableA A  
LEFT JOIN TableB B  
ON A.Key = B.Key  
WHERE B.Key IS NULL
```



```
SELECT <select_list>  
FROM TableA A  
INNER JOIN TableB B  
ON A.Key = B.Key
```



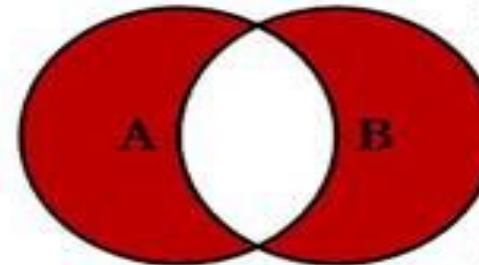
```
SELECT <select_list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.Key = B.Key
```



```
SELECT <select_list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.Key = B.Key  
WHERE A.Key IS NULL
```



```
SELECT <select_list>  
FROM TableA A  
FULL OUTER JOIN TableB B  
ON A.Key = B.Key
```

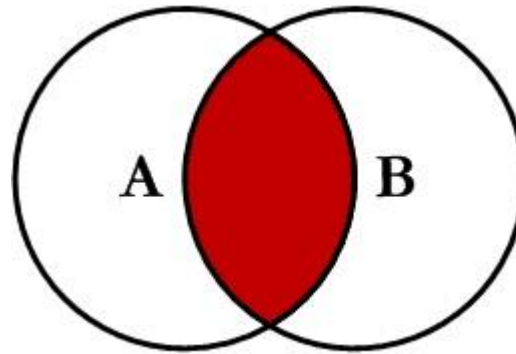


```
SELECT <select_list>  
FROM TableA A  
FULL OUTER JOIN TableB B  
ON A.Key = B.Key  
WHERE A.Key IS NULL  
OR B.Key IS NULL
```

Junções

[INNER] JOIN

- Método de junção que retorna os registros que são comuns às duas tabelas.



Junções

[INNER] JOIN

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
INNER JOIN TabelaB as b  
    on a.Nome = b.Nome;
```



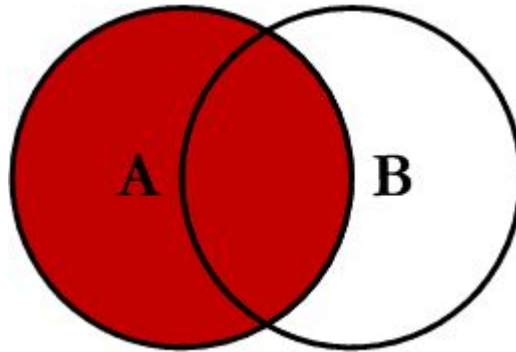
The screenshot shows a database query results window with two tabs: 'Results' and 'Messages'. The 'Results' tab is active, displaying a table with two columns, both labeled 'Nome', and two rows of data. The first row contains 'Luiz' in both columns, and the second row contains 'Femando' in both columns. The cell containing 'Luiz' in the first column is highlighted with a dashed border.

	Nome	Nome
1	Luiz	Luiz
2	Femando	Femando

Junções

LEFT [OUTER] JOIN

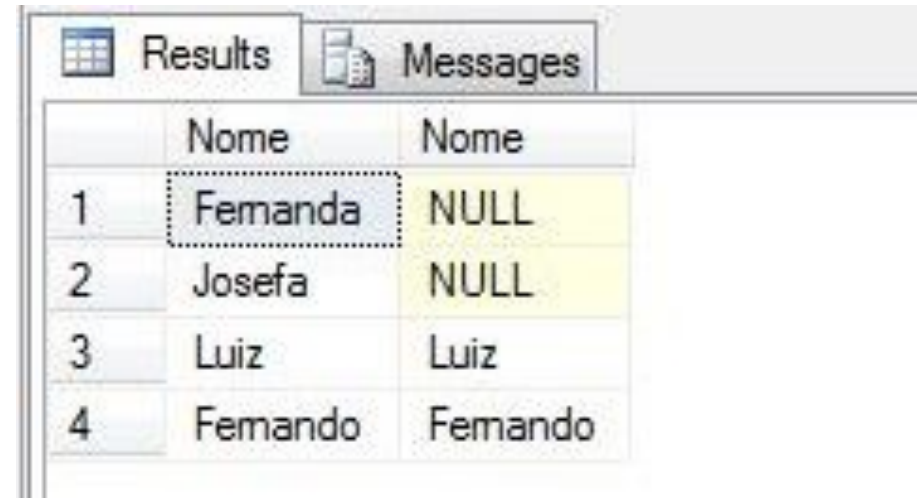
- O Left Join tem como resultado todos os registros que estão na tabela A (mesmo que não estejam na tabela B) e os registros da tabela B que são comuns à tabela A.



Junções

LEFT [OUTER] JOIN

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
LEFT JOIN TabelaB as b  
on a.Nome = b.Nome;
```



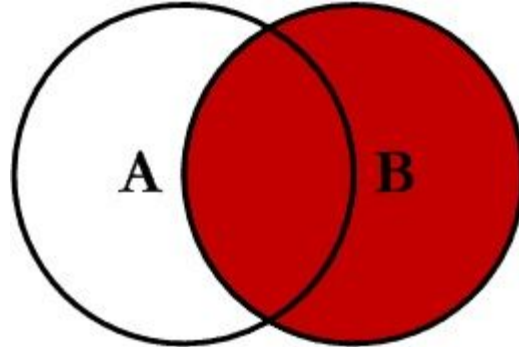
The screenshot shows a database query results window with two tabs: 'Results' and 'Messages'. The 'Results' tab is active, displaying a table with two columns, 'Nome' and 'Nome', and four rows. The first row shows 'Femanda' in the first column and 'NULL' in the second column. The second row shows 'Josefa' in the first column and 'NULL' in the second column. The third row shows 'Luiz' in both columns. The fourth row shows 'Femando' in both columns. The first two rows are highlighted in yellow, indicating they are the result of the LEFT JOIN where no matching record was found in TabelaB.

	Nome	Nome
1	Femanda	NULL
2	Josefa	NULL
3	Luiz	Luiz
4	Femando	Femando

Junções

RIGHT [OUTER] JOIN

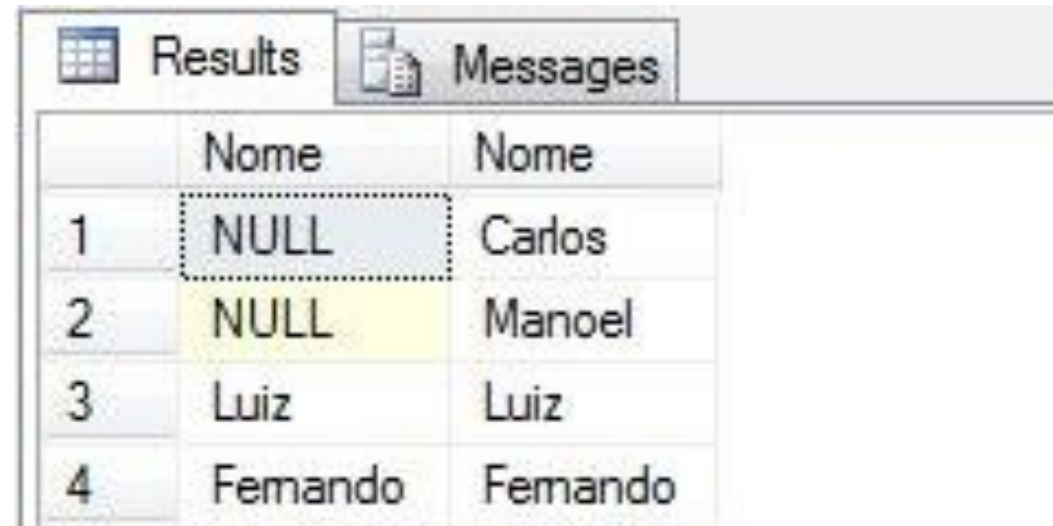
- O Right Join tem como resultado todos os registros que estão na tabela B (mesmo que não estejam na tabela A) e os registros da tabela A que são comuns à tabela B.



Junções

RIGHT [OUTER] JOIN

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
RIGHT JOIN TabelaB as b  
on a.Nome = b.Nome;
```

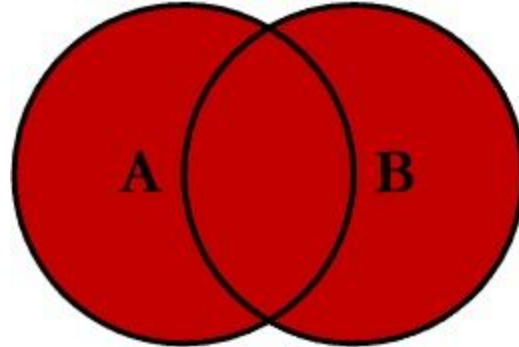


	Nome	Nome
1	NULL	Carlos
2	NULL	Manoel
3	Luiz	Luiz
4	Fernando	Fernando

Junções

FULL [OUTER] JOIN

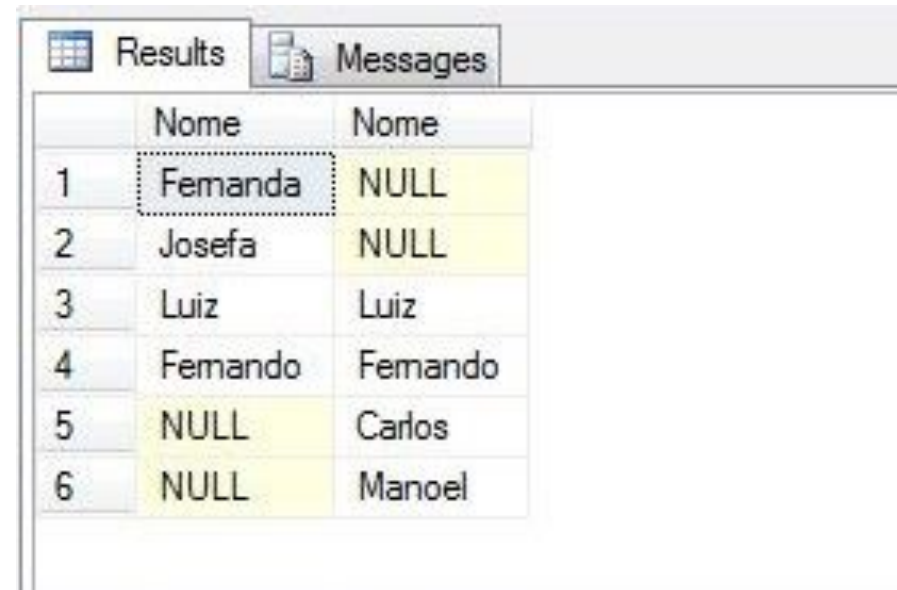
- O Outer Join (também conhecido por Full Outer Join ou Full Join) tem como resultado todos os registros que estão na tabela A e todos os registros da tabela B.



Junções

FULL [OUTER] JOIN (Genérico)

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
FULL OUTER JOIN TabelaB as b  
on a.Nome = b.Nome;
```



The screenshot shows a database query results window with two tabs: 'Results' and 'Messages'. The 'Results' tab is active, displaying a table with two columns, both labeled 'Nome'. The table contains six rows of data. The first row has 'Femanda' in the first column and 'NULL' in the second. The second row has 'Josefa' and 'NULL'. The third row has 'Luiz' and 'Luiz'. The fourth row has 'Femando' and 'Femando'. The fifth row has 'NULL' and 'Carlos'. The sixth row has 'NULL' and 'Manoel'. The first two rows are highlighted in yellow, and the first row is also enclosed in a dashed border.

	Nome	Nome
1	Femanda	NULL
2	Josefa	NULL
3	Luiz	Luiz
4	Femando	Femando
5	NULL	Carlos
6	NULL	Manoel

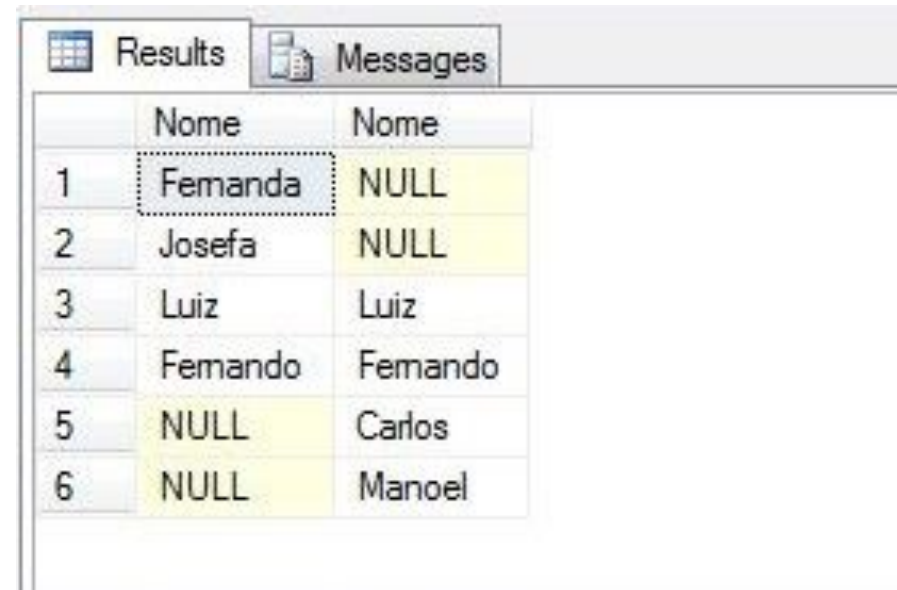
Junções

FULL [OUTER] JOIN (MySQL)

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
LEFT JOIN TabelaB as b  
    on a.Nome = b.Nome;
```

UNION

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
RIGHT JOIN TabelaB as b  
    on a.Nome = b.Nome;
```



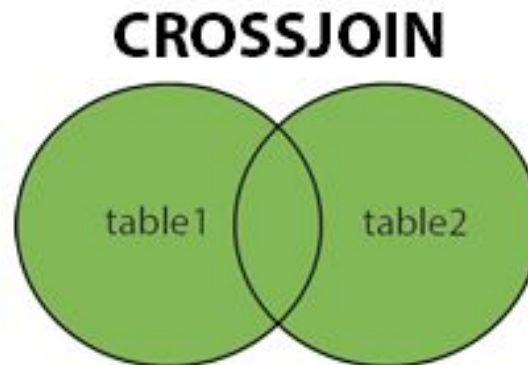
The screenshot shows a database query results window with two tabs: 'Results' and 'Messages'. The 'Results' tab is active, displaying a table with two columns, 'Nome' and 'Nome', and six rows. The first row has 'Femanda' and 'NULL'. The second row has 'Josefa' and 'NULL'. The third row has 'Luiz' and 'Luiz'. The fourth row has 'Femando' and 'Femando'. The fifth row has 'NULL' and 'Carlos'. The sixth row has 'NULL' and 'Manoel'. The first two rows are highlighted in yellow, and the first row is also outlined with a dashed border.

	Nome	Nome
1	Femanda	NULL
2	Josefa	NULL
3	Luiz	Luiz
4	Femando	Femando
5	NULL	Carlos
6	NULL	Manoel

Junções

CROSS JOIN

- A cláusula Cross Join retorna todas as linhas das tabelas por cruzamento, ou seja, para cada linha da tabela esquerda queremos todos os linhas da tabelas direita ou vice-versa. Ele também é chamado de produto cartesiano entre duas tabelas. Porém, para isso é preciso que ambas tenham o campo em comum, para que a ligação exista entre as duas tabelas.



Junções

CROSS JOIN

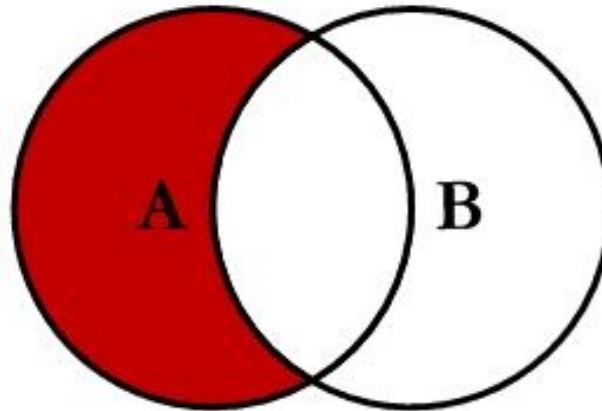
```
SELECT s.store_name, p.product_name  
FROM Stores as s  
CROSS JOIN Products as p;
```

	store_name	product_name
▶	North	iPhone
	South	iPhone
	North	iPad
	South	iPad
	North	Macbook Pro
	South	Macbook Pro

Junções

LEFT [OUTER EXCLUDING] JOIN

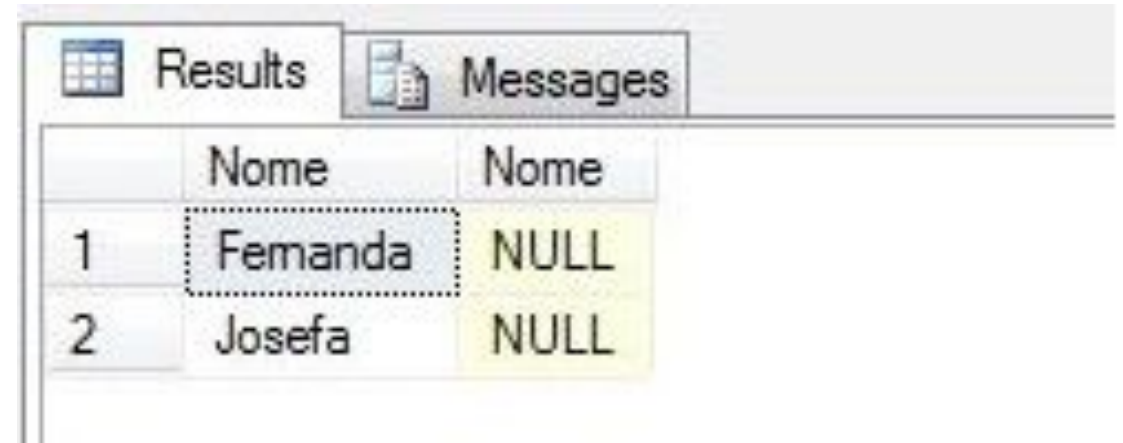
- Left Excluding Join retorna como resultado todos os registros que estão na tabela A e que não estejam na tabela B.



Junções

LEFT [OUTER EXCLUDING] JOIN

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
LEFT JOIN TabelaB as b  
      on a.Nome = b.Nome  
WHERE b.Nome is null;
```

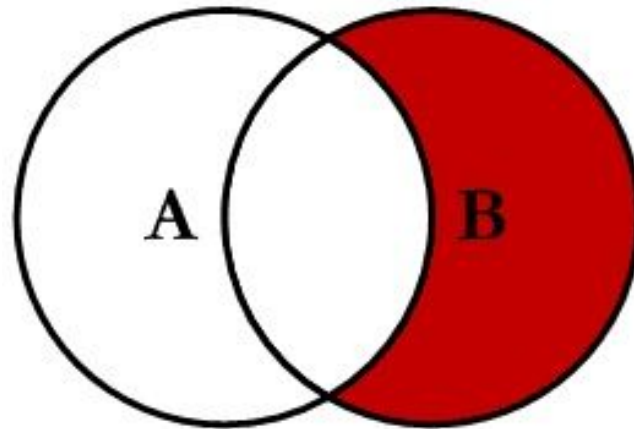


	Nome	Nome
1	Femanda	NULL
2	Josefa	NULL

Junções

RIGHT [OUTER EXCLUDING] JOIN

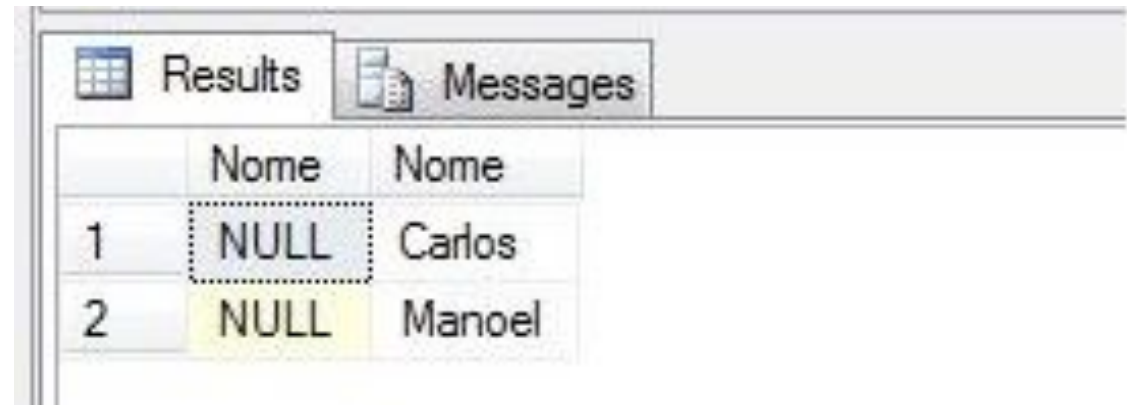
- O Right Excluding Join retorna como resultado todos os registros que estão na tabela B e que não estejam na tabela A.



Junções

RIGHT [OUTER EXCLUDING] JOIN

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
RIGHT JOIN TabelaB as b  
    on a.Nome = b.Nome  
WHERE a.Nome is null;
```

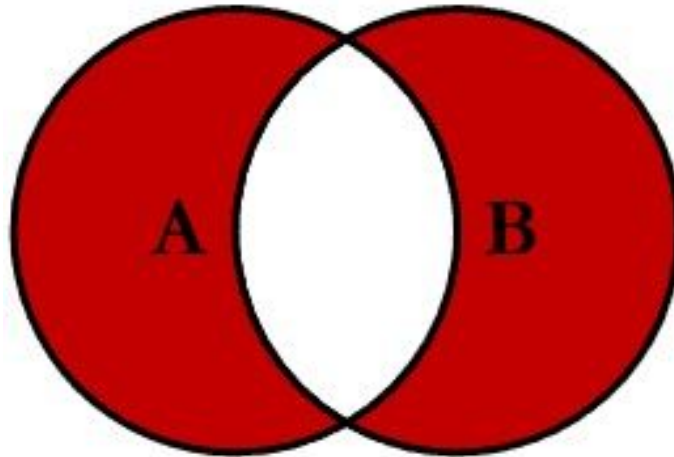


	Nome	Nome
1	NULL	Carlos
2	NULL	Manoel

Junções

FULL [OUTER EXCLUDING] JOIN

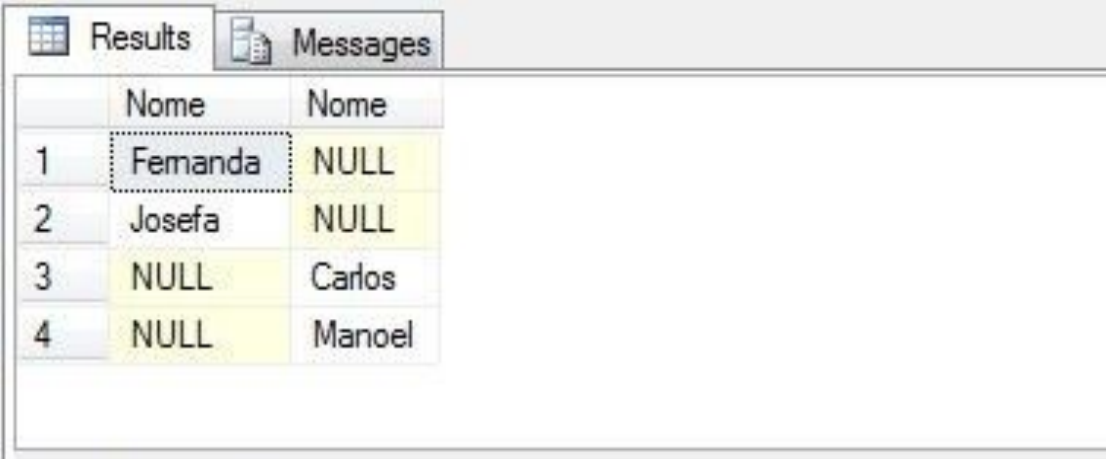
- O Outer Excluding Join tem como resultado todos os registros que estão na tabela B, mas que não estejam na tabela A, e todos os registros que estão na tabela A, mas que não estejam na tabela B.



Junções

FULL [OUTER EXCLUDING] JOIN

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
FULL OUTER JOIN TabelaB as b  
    on a.Nome = b.Nome  
WHERE a.Nome is null or b.Nome is null;
```



The screenshot shows a database query results window with two tabs: 'Results' and 'Messages'. The 'Results' tab is active, displaying a table with two columns, 'Nome' and 'Nome', and four rows. The first row contains 'Fernanda' and 'NULL'. The second row contains 'Josefa' and 'NULL'. The third row contains 'NULL' and 'Carlos'. The fourth row contains 'NULL' and 'Manoel'. The first two rows are highlighted in yellow, and the last two rows are highlighted in light blue. A dashed border is around the cell containing 'Fernanda'.

	Nome	Nome
1	Fernanda	NULL
2	Josefa	NULL
3	NULL	Carlos
4	NULL	Manoel

Junções

Casos especiais:

- **Self Joins:** Junção de uma tabela com ela mesma.
- **Auto Joins:** Junção automática de duas tabelas, de forma implícita.

Junções

- É possível então fazer junções implicitamente, sem utilizar as cláusulas de Junção?
 - Sim.

[INNER] JOIN

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
INNER JOIN TabelaB as b  
    on a.Nome = b.Nome;
```

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a, TabelaB as b  
WHERE a.Nome=b.Nome;
```

LEFT [OUTER] JOIN

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
LEFT JOIN TabelaB as b  
    on a.Nome = b.Nome;
```

```
SELECT a.Nome, NULL  
FROM TabelaA as a  
WHERE NOT EXISTS (  
    SELECT * FROM TabelaB as b  
    WHERE a.Nome=b.Nome)
```

UNION

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a, TabelaB as b  
WHERE a.Nome=b.Nome;
```

RIGHT [OUTER] JOIN

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
RIGHT JOIN TabelaB as b  
on a.Nome = b.Nome;
```

```
SELECT NULL, b.Nome  
FROM TabelaB AS b  
WHERE NOT EXISTS (  
    SELECT * FROM TabelaA as a  
    WHERE a.Nome=b.Nome)
```

UNION

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a, TabelaB as b  
WHERE a.Nome=b.Nome;
```

FULL [OUTER] JOIN

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
LEFT JOIN TabelaB as b  
    on a.Nome = b.Nome;
```

UNION

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
RIGHT JOIN TabelaB as b  
    on a.Nome = b.Nome;
```

```
SELECT a.Nome, NULL  
FROM TabelaA as a  
WHERE NOT EXISTS (  
    SELECT * FROM TabelaB as b  
    WHERE a.Nome=b.Nome)
```

UNION

```
SELECT NULL, b.Nome  
FROM TabelaB AS b  
WHERE NOT EXISTS (  
    SELECT * FROM TabelaA as a  
    WHERE a.Nome=b.Nome)
```

UNION

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a, TabelaB as b  
WHERE a.Nome=b.Nome;
```

LEFT [OUTER EXCLUDING] JOIN

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
LEFT JOIN TabelaB as b  
    on a.Nome = b.Nome;
```

```
SELECT a.Nome, NULL  
FROM TabelaA as a  
WHERE NOT EXISTS (  
    SELECT * FROM TabelaB as b  
    WHERE a.Nome=b.Nome);
```

RIGHT [OUTER EXCLUDING] JOIN

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
RIGHT JOIN TabelaB as b  
on a.Nome = b.Nome;
```

```
SELECT NULL, b.Nome  
FROM TabelaB AS b  
WHERE NOT EXISTS (  
    SELECT * FROM TabelaA as a  
    WHERE a.Nome=b.Nome);
```


FULL [OUTER EXCLUDING] JOIN

```
SELECT a.Nome, b.Nome  
FROM TabelaA as a  
FULL OUTER JOIN TabelaB as b  
on a.Nome = b.Nome;
```

```
SELECT a.Nome, NULL  
FROM TabelaA as a  
WHERE NOT EXISTS (  
    SELECT * FROM TabelaB as b  
    WHERE a.Nome=b.Nome)
```

UNION

```
SELECT NULL, b.Nome  
FROM TabelaB AS b  
WHERE NOT EXISTS (  
    SELECT * FROM TabelaA as a  
    WHERE a.Nome=b.Nome);
```

Subconsultas

- Uma **Subquery** (também conhecida como **Subconsulta** ou **Subselect**) é uma instrução do tipo SELECT dentro de outra instrução SQL. Desta forma, se torna possível efetuar consultas que de outra forma seriam extremamente complicadas ou impossíveis de serem feitas de outra forma.

```
SELECT *  
FROM tabela1 as t1  
WHERE coluna1 IN  
(    SELECT coluna2 FROM tabela2 as t2  
    WHERE t1.id = t2.id  
);
```

Subconsultas

Como utilizar?

- **Subquery como uma nova coluna da consulta** (SELECT AS FIELD).
- **Subquery como filtro de uma consulta** (utilizando IN, EXISTS ou operadores de comparação).
- **Subquery como fonte de dados de uma consulta principal** (SELECT FROM SELECT)

Há quatro formas de criar uma subconsulta em uma instrução SELECT:

- Por meio de uma cláusula WHERE como condição de pesquisa;
- Por meio de uma cláusula HAVING como condição de pesquisa;
- Por meio de uma cláusula FROM para especificação da tabela;
- Por meio de um SELECT para especificação de uma coluna.

Subconsultas

Subquery como uma nova coluna da consulta

```
SELECT
    P.titulo,
    (SELECT COUNT(C.id_projeto)
     FROM comentarios C
     WHERE C.id_projeto = P.id ) as Quantidade_Comentarios
FROM projetos P
GROUP BY (P.id);
```

Subconsultas

Subquery como filtro de uma consulta

```
SELECT P.id, P.titulo, P.data  
FROM projetos P  
WHERE P.id IN  
    (SELECT C.id_projeto  
     FROM comentarios C  
     WHERE P.id = C.id_projeto  
    );
```

Subconsultas

Subquery como filtro de uma consulta

```
SELECT P.id, P.titulo, P.data  
FROM projetos P  
WHERE EXISTS  
    (SELECT C.id_projeto  
     FROM comentarios C  
     WHERE P.id = C.id_projeto  
    );
```

Subconsultas

Subquery como filtro de uma consulta

```
SELECT P.titulo, P.data  
FROM projetos P  
WHERE P.id = (SELECT MAX(LP.id_projeto)  
              FROM likes_por_projeto LP);
```

Subconsultas

Subquery como fonte de dados de uma consulta principal

```
SELECT P.id, P.titulo,  
       (SELECT COUNT(C.id_projeto)  
        FROM comentarios C  
        WHERE C.id_projeto = P.id ) AS Quantidade_Comentarios  
FROM projetos P;
```


Subconsultas

Subquery como fonte de dados de uma consulta principal

```
SELECT F.titulo, F.Quantidade_Comentarios
FROM
    (SELECT P.id, P.titulo,
        (SELECT COUNT(C.id_projeto)
         FROM comentarios C
         WHERE C.id_projeto = P.id ) as Quantidade_Comentarios
     FROM projetos P) as F
WHERE F.Quantidade_Comentarios > 2;
```

Exercícios de Consultas Avançadas - Lista 1

Junções:

- Listar os nomes, código do cliente e o código do pedido para todos os clientes que têm pedidos.
- Listar todos os clientes e respectivos números de pedidos. Caso um cliente não tenha pedidos associados ele também deve aparecer.
- Quais clientes têm pedidos com prazo de entrega superior a 15 dias e pertencem aos estados SP ou RJ?
- Listar os nomes dos clientes e seus respectivos prazos de entrega, ordenados do maior para o menor
- Listar os vendedores (ordenados) que emitiram pedidos com prazos de entrega superiores a 15 dias e tenham salários fixos iguais ou superiores a R\$ 1.000,00.

Subconsultas:

- Que produtos (mostre nome) participam de qualquer pedido com quantidade = 10?
- Quais os vendedores que ganham um salário fixo abaixo da média?
- Quais produtos não estão presentes em nenhum pedido?

Referências

- COUGO, Paulo. Modelagem conceitual e projeto de banco de dados. São Paulo: Elsevier, 2017.
- MACHADO, Felipe Nery Rodrigues; ABREU, Maurício Pereira de. Projeto de banco de dados: uma visão prática. 17. ed. São Paulo, SP: Érica, 2013.
- RAMAKRISHNAN, Raghu; GEHRKE, Johannes. Sistemas de gerenciamento de banco de dados. São Paulo, SP: McGraw-Hill, c2008.

COMPLEMENTAR

- DATE, C. J. Introdução a sistemas de bancos de dados. [8. ed.], 7. tiragem. Rio de Janeiro, RJ: Elsevier, c2004.
- DATE, C. J. Projeto de banco de dados e teoria relacional. São Paulo: Novatec, 2015.
- ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de banco de dados. 6. ed. 3. reimp. São Paulo, SP: Pearson Addison Wesley, 2014.
- OPPEL, Andy; SHELDON, Robert. SQL: um guia para iniciantes. 3. ed. Rio de Janeiro, RJ: Ciência Moderna, 2009.
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de banco de dados. 6 ed. São Paulo, SP: Makron books, 2012.

Consultas Avançadas: Junções e Subconsultas

Prof. Cloves Rocha

Ciência da Computação - CC